PYTHON LISTS

Programming for Data Science

Lists

- Lists are used to store multiple items in a single variable.
- Lists are one of 4 built-in data types in Python used to store collections of data, the other 3 are Tuple, Set, and Dictionary, all with different qualities and usage.
- Lists are created using square brackets “[]” or the list() constructor.

```
mylist = ["apple", "banana", "cherry"]

thislist = list(("apple", "banana", "cherry"))
```

Lists Items

- List items are **ordered**, **changeable**, and allow **duplicate** values.
- List items are indexed, the first item has index [0], the second item has index [1] etc.

```
thislist = ["apple", "banana", "cherry", "apple", "cherry"]
print(len(thislist))
```

Lists - Data Types

```
list1 = ["apple", "banana", "cherry"]
list2 = [1, 5, 7, 9, 3]
list3 = [True, False, False]
list4 = ["abc", 34, True, 40, "male"]

mylist = ["apple", "banana", "cherry"]
print(type(mylist))   # <class 'list'>
```

Access Items

- List items are indexed and you can access them by referring to the index number

```
thislist = ["apple", "banana", "cherry", "orange",
"kiwi", "melon", "mango"]

print(thislist[1])
print(thislist[-1])
print(thislist[2:5])
print(thislist[:4])
print(thislist[2:])
print(thislist[-4:-1])
```

Access Items

- List items are indexed and you can access them by referring to the index number

```
thislist = ["apple", "banana", "cherry", "orange",
"kiwi", "melon", "mango"]

print(thislist[1])
print(thislist[-1])
print(thislist[2:5])
print(thislist[:4])
print(thislist[2:])
print(thislist[-4:-1])
```

Output

```
banana
mango
['cherry', 'orange', 'kiwi']
['apple', 'banana', 'cherry', 'orange']
['cherry', 'orange', 'kiwi', 'melon', 'mango']
['orange', 'kiwi', 'melon']
```

Check if Item Exists

- To determine if a specified item is present in a list use the in keyword.

```
thislist = ["apple", "banana", "cherry"]
if "apple" in thislist:
  print("Yes, 'apple' is in the fruits list")
```

Output

```
Yes, 'apple' is in the fruits list
```

Change List Items

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi",
"mango"]
thislist[1] = "blackberry"
thislist[1:3] = ["blackberry", "watermelon"]
thislist[1:2] = ["blackberry", "watermelon"]
thislist[1:3] = ["watermelon"]
```

Change List Items

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi",
"mango"]
```

Output

```
thislist[1]= ["apple", "blackberry", "cherry", "orange", "kiwi","mango"]
thislist[1:3]= ["apple", "blackberry", "watermelon", "orange", "kiwi", "mango"]
thislist[1:2]= ["apple", "blackberry", "watermelon", "watermelon", "orange", "kiwi",
"mango"]
thislist[1:3]= ["apple", "watermelon", "watermelon", "orange", "kiwi", "mango"]
```

Add List Items

```
thislist = ["apple", "banana", "cherry"]
thislist.insert(1, "orange1")
#print(thislist)
thislist.append("orange2")
print(thislist)
```

Output

```
['apple', 'orange1', 'banana', 'cherry', 'orange2']
```

Add List Items

```
thislist = ["apple", "banana", "cherry"]
tropical = ["mango", "pineapple"]
thislist.extend(tropical )
print(thislist)
```

```
thislist = ["apple", "banana", "cherry"]
tropical = ("mango", "pineapple")
thislist.extend(tropical)
print(thislist)
```

Output

```
['apple', 'banana', 'cherry', 'mango', 'pineapple']
```

Remove List Items

```
thislist = ["apple", "banana", "cherry"]
thislist.remove("banana")
```

If there are more than one item with the specified value, the `remove()` method removes the first occurrence.

```
thislist = ["apple", "banana", "cherry"]
thislist.pop(1)
thislist.pop()
```

If you do not specify the index, the `pop()` method removes the last item.

Remove List Items

```
thislist = ["apple", "banana", "cherry"]
del thislist[0]
del thislist
```

The `del` keyword can delete the list completely. Using this list will raise exception.

```
thislist = ["apple", "banana", "cherry"]
thislist.clear()
```

The list still remains, but it has no content.

Loop Lists

```
thislist = ["apple", "banana", "cherry"]
for x in thislist:
  print(x)
```

```
thislist = ["apple", "banana", "cherry"]
for i in range(len(thislist)):
  print(thislist[i])
```

```
thislist = ["apple", "banana", "cherry"]
i = 0
while i < len(thislist):
  print(thislist[i])
  i = i + 1
```

Output

```
apple
banana
cherry
```

List Comprehension

- List comprehension offers a shorter syntax when you want to create a new list based on the values of an existing list.

```
newlist = [expression for item in iterable if condition == True]
```

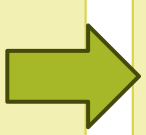
- The return value is a new list, leaving the old list unchanged.

List Comprehension

```
fruits = ["apple", "banana",
"cherry", "kiwi", "mango"]
newlist = []

for x in fruits:
  if "a" in x:
    newlist.append(x)

print(newlist)
```



```
fruits = ["apple", "banana", "cherry",
"kiwi", "mango"]

newlist = [x for x in fruits if "a" in x]

print(newlist)
```

Output

```
['apple', 'banana', 'mango']
```

List Comprehension

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
newlist = [x for x in fruits]
newlist = [x for x in fruits if x != "apple"]

newlist = [x.upper() for x in fruits]
newlist = ['hello' for x in fruits]
newlist = [x if x != "banana" else "orange" for x in fruits]
```

```
newlist = [x for x in range(10)]
newlist = [x for x in range(10) if x < 5]
```

Sort Lists

List objects have a `sort()` method that will sort the list alphanumerically, ascending, by default.

```
thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]
thislist.sort()
print(thislist)
```

Output

```
['banana', 'kiwi', 'mango', 'orange', 'pineapple']
```

```
thislist = [100, 50, 65, 82, 23]
thislist.sort()
print(thislist)
```

Output

```
[23, 50, 65, 82, 100]
```

Sort Lists

To sort descending, use the keyword argument `reverse = True`

```
thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]
thislist.sort(reverse = True)
print(thislist)
```

Output

```
['pineapple', 'orange', 'mango', 'kiwi', 'banana']
```

```
thislist = [100, 50, 65, 82, 23]
thislist.sort(reverse = True)
print(thislist)
```

Output

```
[100, 82, 65, 50, 23]
```

Sort Lists - Customize Sort Function

`sort()` function can be customized by using the keyword argument `key = function`.

The function will return a number that will be used to sort the list (the lowest number first)

```
def myfunc(n):
  return abs(n - 50)

thislist = [100, 50, 65, 82, 23]
thislist.sort(key = myfunc)
print(thislist)
```

Output

```
[50, 65, 23, 82, 100]
```

Sort Lists

We can use built-in functions as key functions when sorting a list.

```
thislist = ["Banana", "orange", "Kiwi", "cherry"]
thislist.sort(key = str.lower)
print(thislist)
```

Output

```
['banana', 'cherry', 'kiwi', 'orange']
```

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]
thislist.reverse()
print(thislist)
```

Output

```
['cherry', 'Kiwi', 'Orange', 'banana']
```

Copy Lists

We cannot copy a list simply by typing `list2 = list1`, because: `list2` will only be a reference to `list1`, and changes made in `list1` will automatically also be made in `list2`. We can use the built-in List method `copy()` to copy a list.

```
thislist = ["apple", "banana", "cherry"]
mylist1 = thislist.copy()
mylist2 = list(thislist)
mylist3 = thislist[:]
print(mylist1)
print(mylist2)
print(mylist3)
```

Output

```
['apple', 'banana', 'cherry']
['apple', 'banana', 'cherry']
['apple', 'banana', 'cherry']
```

Join Lists

```
list1 = ["a", "b", "c"]
list2 = [1, 2, 3]
list3 = [4, 5, 6]

list4 = list1 + list2
print("list4: ", list4)

for x in list2:
  list1.append(x)
print("list1: ", list1)

# the loop is same as the following
list2.extend(list3)
print("list2: ", list2)
```

Output

```
list4:  ['a', 'b', 'c', 1, 2, 3]
list1:  ['a', 'b', 'c', 1, 2, 3]
list2:  [1, 2, 3, 4, 5, 6]
```

Enumerate

```
list1 = ["a", "b", "c"]

for x in list1:
  print(x)

for indx, x in enumerate(list1):
  print(indx, "=>", x)
```

Output

```
a
b
c
0 => a
1 => b
2 => c
```

Questions?

Thank you!